The Architecture of the Strangler Pattern

A Layered Approach

The Strangler Pattern adopts a layered approach in its architecture. Imagine peeling an onion layer by layer, with each layer representing a part of the legacy system being replaced by the new system.

Facade Interface: The Gatekeeper

The central component in the Strangler Pattern's architecture is the Facade Interface. This is the gatekeeper that decides whether a request should be handled by the new system or the legacy system. Let's think of it as a traffic controller at an intersection, guiding cars (or in our case, requests) down the correct road.

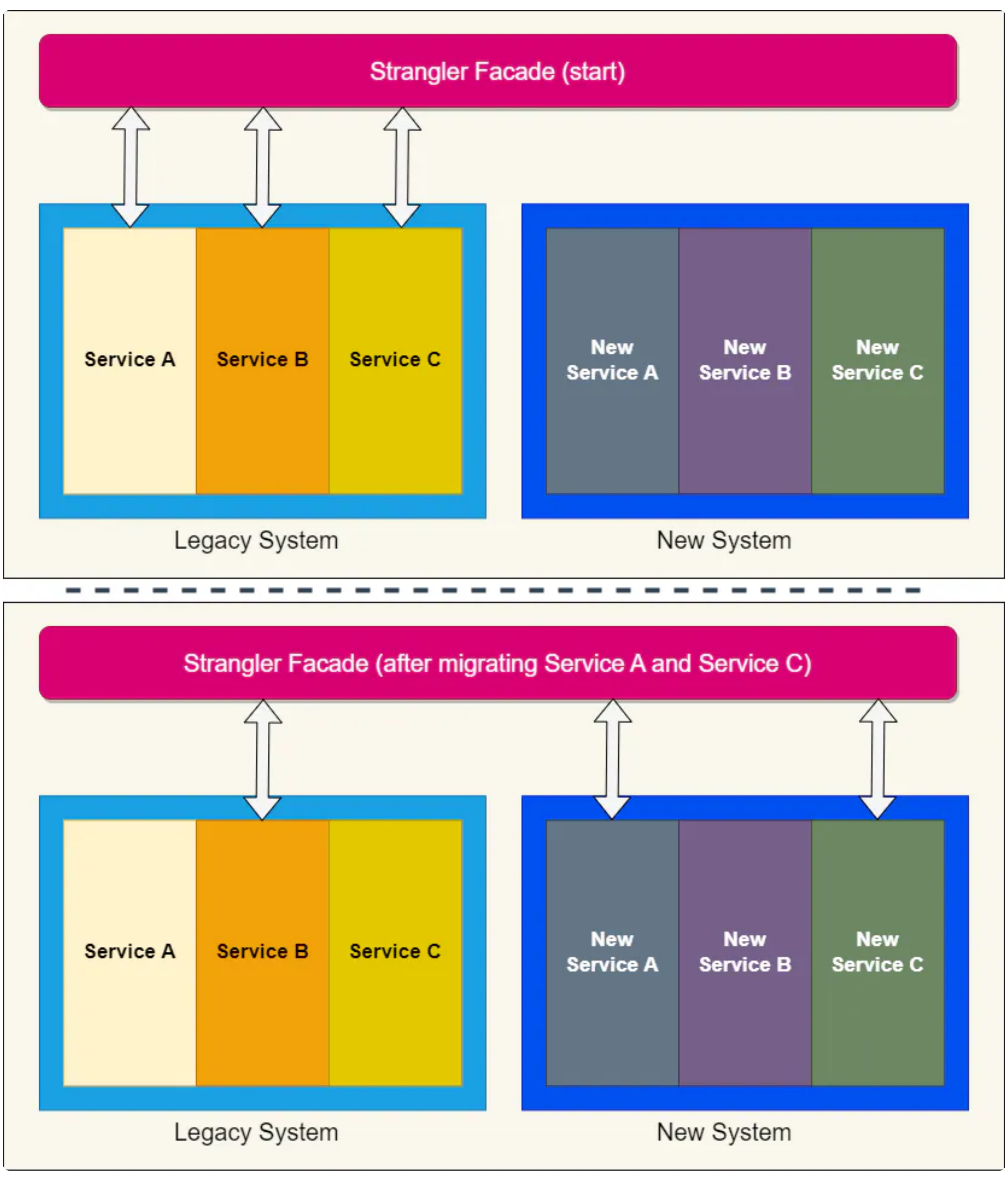
When a request arrives, the Facade Interface checks if the requested functionality has been migrated to the new system. If it has, the request is directed to the new system. If not, the request continues to be handled by the legacy system.

Phased Migration: One Step at a Time

The migration from the legacy system to the new system doesn't happen all at once. It happens in a step-by-step process.

The migration starts by identifying a piece of functionality that can be easily isolated and redirected from the legacy system to the new system. This piece of functionality is then rebuilt in the new system.

Once it's fully tested and ready to go live, the Facade Interface starts redirecting requests for this functionality to the new system, while still sending the remaining requests to the legacy system. This gradual shift of responsibilities from the old system to the new one forms the crux of the Strangler Pattern.



Strangler Pattern - Step by step migration

Strangler Fig: An Example: Imagine we have an old system for user management and we want to replace the "getUserDetails" function with a new service. We might use a simple router to decide which service to use:

```
class UserServiceRouter:

    def getUserDetails(self, userID):
        if self.useNewService(userID):
            return NewUserService().getUserDetails(userID)
        else:
            return OldUserService().getUserDetails(userID)

    def useNewService(self, userID):
        # Logic to decide which service to use.
        # For simplicity, let's say we use the new service for even userIDs.
        return userID % 2 == 0

class OldUserService:
    def getUserDetails(self, userID):
        # Old way of fetching user details
        pass

class NewUserService:
    def getUserDetails(self, userID):
        # New, improved way of fetching user details
        pass
```

As we become confident in `NewUserService`, we can adjust `useNewService` logic to send more traffic to the new service until `OldUserService` is no longer used.

← Previous
The Strangler Pattern: A Solution

Next →
Strangler Pattern: A Detailed Example

☐ Mark as Completed